

CONTENTS

Módulo 6 — Ensamblador Intel 8086

Direccionamiento segmentado, registros, modos de direccionamiento, cartilla de instrucciones y compilación a mano de funciones recursivas

0. Por qué existe este tema (y por qué te está costando el examen)

1. Direccionamiento segmentado: cómo se calcula la dirección física

1.1 Definición

1.2 Procedimiento paso a paso

1.3 Ejemplo resuelto 1

1.4 Ejemplo resuelto 2 (con acarreo y solape de segmentos)

1.5 Ejemplo del caso límite (para no perder puntos en el examen)

1.6 Tabla resumen

2. Set de registros del 8086

2.1 Registros de datos (16 bits, con acceso a sus mitades de 8 bits)

2.2 Registros de segmento

2.3 Punteros de pila, índices y puntero de instrucción

2.4 Registro de FLAGS

3. Modos de direccionamiento del 8086

3.1 Ejemplo concreto: [BX]

3.2 Ejemplo concreto: [BX+SI]

3.3 Ejemplo concreto: [BP+DI+desplazamiento]

4. Cartilla de instrucciones más importantes

4.1 Transferencia y aritmética

4.2 Lógicas y de desplazamiento

4.3 Saltos condicionales (y a qué flags responden)

4.4 Pila y control de flujo (las instrucciones del ejercicio estrella)

5. Directivas de ensamblador

6. EL EJERCICIO MÁS IMPORTANTE DEL CURSO: compilar a mano una función recursiva

6.1 El problema en alto nivel

6.2 Código ensamblador completo, línea por línea	
6.3 Cálculo del consumo máximo de stack (n=5)	
6.4 Segundo ejemplo resuelto y verificado: fibonacci con parámetros por registro (protocolo alternativo)	
7. Por qué [BX+SI] (direccionamiento indexado) es clave para recorrer arreglos y arrays de structs	
7.1 La idea	
7.2 Ejemplo: array de structs (Práctico 7, Ejercicio 7 / Práctico 8, árbol)	
7.3 Ejemplo: matriz de matrices (Práctico 7, Ejercicio 9)	
Ejercicios de repaso rápido (con respuesta)	
Resumen ultra-rápido (2 minutos)	

Módulo 6 — Ensamblador Intel 8086

DIRECCIONAMIENTO SEGMENTADO, REGISTROS, MODOS DE DIRECCIONAMIENTO, CARTILLA DE INSTRUCCIONES Y COMPILACIÓN A MANO DE FUNCIONES RECURSIVAS

0. Por qué existe este tema (y por qué te está costando el examen)

El 8086 es una CPU de 16 bits que necesita direccionar 1 MB de memoria (20 bits de dirección física), pero sus registros son de 16 bits (direccionan solo 64 KB). La solución de Intel fue el **direccionamiento segmentado**: toda dirección de memoria se expresa como un par SEGMENTO:DESPLAZAMIENTO, y el hardware combina ambos para producir una dirección física de 20 bits.

Esto no es solo teoría de arranque: **todo lo demás del módulo depende de entender esto primero**, porque:

- Cada acceso a memoria en una instrucción (`MOV AX, [BX]`, `PUSH ES:[DI]`, etc.) usa un segmento implícito o explícito, y si no sabés cuál es, no podés calcular direcciones efectivas ni interpretar el código.
- El ejercicio estrella del curso —**compilar a mano una función recursiva**— exige entender simultáneamente: cómo se pasan parámetros (registros o pila), cómo la pila crece hacia direcciones bajas, cómo se arma un frame con `BP`, y cómo cada `CALL / PUSH` consume bytes de un segmento de pila que tiene tamaño finito. Si fallás en cualquiera de esos eslabones, la cuenta final de stack da mal.

Peso en exámenes: este tema (función recursiva compilada a mano + cálculo de stack) aparece en 6 de los 13 exámenes analizados. Es, con diferencia, el tema con mayor retorno por hora de estudio. Tolerancia cero significa: cada `PUSH` tiene su `POP` en el orden correcto, cada frame de `BP` se arma y se desarma, y la cuenta de bytes se verifica sumando, no “a ojo”.

Convención de este documento: los números en base 16 se escriben con sufijo `h` o prefijo `0x` indistintamente (`0800h` = `0x0800`). Usamos direccionamiento **near** (llamadas dentro del mismo segmento de código) salvo que se indique lo contrario, porque es el caso que trabajan los prácticos 7 y 8.

1. Direccionamiento segmentado: cómo se calcula la dirección física

1.1 DEFINICIÓN

Una **dirección segmentada** (o dirección lógica) se escribe `SEGMENTO:DESPLAZAMIENTO` y NO es la dirección real de memoria: es una fórmula para calcularla. El hardware del 8086 la transforma en una **dirección física** de 20 bits mediante:

Dirección física = SEGMENTO × 16 + DESPLAZAMIENTO

(equivalente a correr el segmento 4 bits a la izquierda — desplazar un dígito hexadecimal — y sumarle el desplazamiento)

El registro de segmento indica el comienzo de un “párrafo” de 16 bytes; el desplazamiento (offset) indica cuántos bytes hay que avanzar desde ahí. Como el segmento se multiplica por 16, dos direcciones segmentadas distintas pueden apuntar a la **misma dirección física** (ver 1.4).

1.2 PROCEDIMIENTO PASO A PASO

1. Tomar el valor del registro de segmento (16 bits) y multiplicarlo por 16, es decir, agregarle un `0` en hexadecimal a la derecha (desplazar 4 bits).
2. Sumarle el desplazamiento (16 bits) tal cual.
3. El resultado es la dirección física de 20 bits (rango `0x00000` a `0xFFFFF`, 1 MB total).

1.3 EJEMPLO RESUELTO 1

Dirección segmentada: `0800h : 0100h` (esta es justamente la dirección que pide el Ejercicio 1 del Práctico 7 para volcar registros en caso de overflow).

```

Segmento × 16 : 0800h × 10h = 08000h
+ Desplazamiento:      + 0100h
-----
Dirección física:      08100h

```

Verificación (aritmética hex explícita):

0800h = 2048 decimal. $2048 \times 16 = 32768 = 8000h$.

$8000h + 100h = 8100h$.

Confirmado con cálculo independiente en shell: `0x0800*16+0x0100 = 0x8100`. ✓

1.4 EJEMPLO RESUELTO 2 (CON ACARREO Y SOLAPE DE SEGMENTOS)

Dirección segmentada: `1234h : 5678h`.

```

Segmento × 16 : 1234h × 10h = 12340h
+ Desplazamiento:      + 5678h
-----
Dirección física:      179B8h

```

Verificación: $1234h \times 16 = 12340h$ (correr un dígito hex). $12340h + 5678h = 179B8h$. Confirmado por cálculo independiente:

`0x1234*16+0x5678 = 0x179B8`. ✓

Ahora comparemos con la dirección segmentada `1793h : 0088h`:

```

1793h × 10h = 17930h
17930h + 0088h = 179B8h  <-- ¡la MISMA dirección física!

```

Error frecuente: pensar que segmento y desplazamiento identifican una dirección física de manera única. Es falso: como el segmento se multiplica por 16, existen 4096 pares SEGMENTO:DESPLAZAMIENTO distintos que producen la misma dirección física. Esto es exactamente la “superposición de segmentos” que pregunta la teoría del Práctico 7(d): si dos segmentos se solapan, escribir a través de uno puede modificar (sin darte cuenta) datos que “pertenecen” al otro segmento, porque en el fondo es la misma memoria física.

1.5 EJEMPLO DEL CASO LÍMITE (PARA NO PERDER PUNTOS EN EL EXAMEN)

Segmento `2000h`, desplazamiento máximo `FFFFh`:

```

2000h × 10h = 20000h
20000h + FFFFh = 2FFFFh

```

Esto muestra que, aun con el mayor desplazamiento posible, la dirección física sigue estando dentro del segmento “extendido” de 64 KB que arranca en `20000h` — el offset nunca genera acarreo hacia el segmento en el 8086 real (no hay wrap-around en la suma dentro del rango de 20 bits salvo casos de “wrap” documentados en el extremo superior de memoria, que quedan fuera del alcance de este curso).

1.6 TABLA RESUMEN

Segmento	Offset	Segmento×16	Dirección física
0800h	0100h	08000h	08100h
1234h	5678h	12340h	179B8h
1793h	0088h	17930h	179B8h (igual a la anterior)
2000h	FFFFh	20000h	2FFFFh
1000h	0010h	10000h	10010h

Error frecuente #2: sumar segmento y desplazamiento sin multiplicar el segmento por 16 primero (es decir, hacer `SEG + OFF` en vez de `SEG×16 + OFF`). Esto da un resultado completamente distinto y es el error más común en los parciales cuando el estudiante tiene apuro. Regla mnemotécnica: “agregale un cero hex al segmento antes de sumar”.

2. Set de registros del 8086

2.1 REGISTROS DE DATOS (16 BITS, CON ACCESO A SUS MITADES DE 8 BITS)

Registro 16 bits	Mitad alta (8 bits)	Mitad baja (8 bits)	Uso típico / implícito
AX (Accumulator)	AH	AL	Acumulador; operando implícito de <code>MUL</code> , <code>DIV</code> , <code>IMUL</code> , <code>IDIV</code> ; retorno de funciones por convención
BX (Base)	BH	BL	Único registro de propósito general utilizable como puntero base de memoria junto con BP
CX (Counter)	CH	CL	Contador implícito de <code>LOOP</code> , de rotaciones/desplazamientos variables (<code>SHL/SHR</code> , ..., <code>CL</code>)
DX (Data)	DH	DL	Extensión de AX en <code>MUL</code> / <code>DIV</code> de 16 bits (par DX:AX); puerto implícito en E/S

AH/AL no son “otros” registros: son la misma celda física que AX vista como dos mitades. Escribir en AL modifica los 8 bits bajos de AX y deja AH intacto (y viceversa). Esto es clave para instrucciones como `MOV BYTE PTR [DI], AL` en rutinas de manejo de strings/bytes.

2.2 REGISTROS DE SEGMENTO

Registro	Segmento que referencia
CS (Code Segment)	Segmento del código en ejecución. Junto con IP forma CS:IP, la dirección de la próxima instrucción
DS (Data Segment)	Segmento por defecto para operandos de memoria (variables)
SS (Stack Segment)	Segmento de la pila. Junto con SP forma SS:SP
ES (Extra Segment)	Segmento auxiliar, usado explícitamente (p. ej. para un segundo buffer, un array “far”, strings)

2.3 PUNTEROS DE PILA, ÍNDICES Y PUNTERO DE INSTRUCCIÓN

Registro	Significado
SP (Stack Pointer)	Desplazamiento (dentro de SS) del tope actual de la pila. Se decrementa en <code>PUSH</code> , se incrementa en <code>POP</code>
BP (Base Pointer)	Desplazamiento (dentro de SS por defecto) usado como base del <i>frame</i> de una función para acceder a parámetros y variables locales en la pila
SI (Source Index)	Índice/puntero fuente, típico en recorridos de arreglos/strings (junto a DS)
DI (Destination Index)	Índice/puntero destino, típico junto a ES
IP (Instruction Pointer)	Desplazamiento (dentro de CS) de la próxima instrucción a ejecutar. No es accesible directamente con <code>MOV</code> ; se modifica con saltos, <code>CALL</code> / <code>RET</code> , interrupciones

2.4 REGISTRO DE FLAGS

El 8086 tiene un registro de banderas de 16 bits. Las banderas **aritméticas** relevantes para este curso son:

Flag	Nombre	Se activa cuando...
ZF	Zero Flag	El resultado de la operación es 0
SF	Sign Flag	El bit más significativo del resultado es 1 (resultado "negativo" interpretado en signo)
CF	Carry Flag	Hubo acarreo (suma) o préstamo (resta) fuera del bit más significativo, interpretando los operandos como sin signo
OF	Overflow Flag	El resultado no entra en el rango representable interpretando los operandos con signo (p. ej. sumar dos positivos y obtener un negativo)
AF	Auxiliary Flag	Acarreo/préstamo entre el nibble bajo y el alto (uso interno, casi no se pregunta)
PF	Parity Flag	La cantidad de bits en 1 del byte bajo del resultado es par (uso casi nulo en este curso)

CF y OF responden preguntas distintas: CF dice si hubo overflow **sin signo** (interpretás los bits como magnitudes positivas), OF dice si hubo overflow **con signo** (interpretás con complemento a 2). Un mismo resultado binario puede tener CF=1 y OF=0, o viceversa, según qué le pediste calcular al programa. Esto es literalmente el Ejercicio 1 del Práctico 7: detectar overflow de una suma (con signo → mirar OF).

3. Modos de direccionamiento del 8086

Modo	Sintaxis ejemplo	¿Cómo se calcula la dirección efectiva (EA)?
Registro	<code>MOV AX, BX</code>	No hay memoria involucrada; el operando ES el registro
Inmediato	<code>MOV AX, 5</code>	El valor viene codificado en la instrucción misma; no hay EA
Directo a memoria	<code>MOV AX, [1000h]</code>	EA = 1000h (constante). Segmento por defecto DS
Indirecto con registro base	<code>MOV AX, [BX]</code>	EA = contenido de BX. Segmento por defecto DS (o SS si el registro base es BP)
Indirecto con registro índice	<code>MOV AX, [SI]</code>	EA = contenido de SI. Segmento por defecto DS
Base + índice	<code>MOV AX, [BX+SI]</code>	EA = BX + SI
Base + índice + desplazamiento	<code>MOV AX, [BP+DI+4]</code>	EA = BP + DI + 4. Segmento por defecto SS (porque interviene BP)

3.1 EJEMPLO CONCRETO: `[BX]`

Si `DS = 3000h` y `BX = 0050h`, la instrucción `MOV AX, [BX]` accede a:

EA (desplazamiento) = `BX = 0050h`
 Dirección física = `DS×16 + EA = 3000h×10h + 0050h = 30000h + 0050h = 30050h`

3.2 EJEMPLO CONCRETO: `[BX+SI]`

Con `DS = 3000h`, `BX = 0050h`, `SI = 0004h`:

EA = `BX + SI = 0050h + 0004h = 0054h`
 Dirección física = `30000h + 0054h = 30054h`

3.3 EJEMPLO CONCRETO: `[BP+DI+DESPLAZAMIENTO]`

Con `SS = 4000h`, `BP = 0010h`, `DI = 0002h`, desplazamiento constante `6`:

EA = `BP + DI + 6 = 0010h + 0002h + 0006h = 0018h`
 Dirección física = `SS×16 + EA = 40000h + 0018h = 40018h` (usa SS por defecto, no DS, por intervenir BP)

Regla práctica para saber qué segmento se usa por defecto: **"BP habla con la pila (SS), todo lo demás habla con datos (DS)"**, salvo que la instrucción tenga un prefijo explícito de segmento (`ES:[DI]`, `SS:[BX]`, etc.), en cuyo caso ese prefijo manda siempre.

Error frecuente: olvidar el segmento por defecto y calcular la dirección física con DS cuando el registro base es BP (debería ser SS). Este es exactamente el motivo por el cual, en el Ejercicio 4 del Práctico 8 (función `profundidad`), el acceso al resultado de retorno se hace con `SS:[BP+6]` explícitamente: el frame vive en la pila, y BP apunta dentro de SS.

4. Cartilla de instrucciones más importantes

"Afecta flags" lista solo ZF/SF/CF/OF (las relevantes para este curso). Cuando una instrucción no afecta flags, se indica "ninguno".

4.1 TRANSFERENCIA Y ARITMÉTICA

Instrucción	Efecto	Flags que afecta
<code>MOV dest, src</code>	$dest \leftarrow src$	ninguno
<code>ADD dest, src</code>	$dest \leftarrow dest + src$	ZF, SF, CF, OF
<code>SUB dest, src</code>	$dest \leftarrow dest - src$	ZF, SF, CF, OF
<code>MUL src</code>	(sin signo) $AX \leftarrow AL \times src$ (8 bits) o $DX:AX \leftarrow AX \times src$ (16 bits)	CF y OF (=1 si la mitad alta del resultado es $\neq 0$, es decir, no entra en el registro simple)
<code>IMUL src</code>	(con signo) igual que MUL pero interpretando con signo	CF y OF (=1 si el resultado no entra en el registro simple, con signo)
<code>DIV src</code>	(sin signo) $AX/src \rightarrow$ cociente en AL, resto en AH (8 bits) o $DX:AX/src \rightarrow$ cociente en AX, resto en DX (16 bits)	indefinidos (no confiar en flags tras DIV)
<code>IDIV src</code>	igual que DIV pero con signo	indefinidos

4.2 LÓGICAS Y DE DESPLAZAMIENTO

Instrucción	Efecto	Flags que afecta
<code>AND dest, src</code>	$dest \leftarrow dest \text{ AND } src$ (bit a bit)	ZF, SF; CF=0, OF=0 siempre
<code>OR dest, src</code>	$dest \leftarrow dest \text{ OR } src$	ZF, SF; CF=0, OF=0
<code>XOR dest, src</code>	$dest \leftarrow dest \text{ XOR } src$	ZF, SF; CF=0, OF=0
<code>NOT dest</code>	$dest \leftarrow$ complemento a 1 de dest (invierte todos los bits)	ninguno (no toca flags)
<code>SHL dest, cnt</code>	Desplaza bits a la izquierda, entra 0 por la derecha	CF = último bit que salió por la izquierda; OF definido solo si $cnt=1$; ZF, SF
<code>SHR dest, cnt</code>	Desplaza bits a la derecha, entra 0 por la izquierda	CF = último bit que salió por la derecha; OF definido solo si $cnt=1$; ZF, SF

Error frecuente: usar `XOR AX, AX` para poner AX en 0 (truco válido y muy usado, como en `busquedaBin` del Práctico 8: `XOR CL, CL`) y después asumir que CF quedó en un valor "de la suma anterior". XOR siempre pone CF=0 y OF=0, así que es seguro, pero no confundir esta instrucción lógica con una resta: `XOR` nunca "toma prestado".

4.3 SALTOS CONDICIONALES (Y A QUÉ FLAGS RESPONDEN)

Instrucción	Salta si...	Uso típico
<code>JE</code> / <code>JZ</code>	ZF=1	Igualdad
<code>JNE</code> / <code>JNZ</code>	ZF=0	Desigualdad
<code>JG</code> / <code>JNLE</code>	ZF=0 y SF=OF	Mayor que, con signo
<code>JGE</code> / <code>JNL</code>	SF=OF	Mayor o igual, con signo
<code>JL</code> / <code>JNGE</code>	SF≠OF	Menor que, con signo
<code>JLE</code> / <code>JNG</code>	ZF=1 o SF≠OF	Menor o igual, con signo
<code>JA</code> / <code>JNBE</code>	CF=0 y ZF=0	Mayor que, sin signo
<code>JAE</code> / <code>JNB</code> / <code>JNC</code>	CF=0	Mayor o igual, sin signo
<code>JB</code> / <code>JNAE</code> / <code>JC</code>	CF=1	Menor que, sin signo
<code>JBE</code> / <code>JNA</code>	CF=1 o ZF=1	Menor o igual, sin signo
<code>JO</code> / <code>JNO</code>	OF=1 / OF=0	Overflow con signo
<code>JS</code> / <code>JNS</code>	SF=1 / SF=0	Resultado negativo
<code>JCXZ</code>	CX=0	No mira flags, mira CX directamente

Error frecuente #1 del curso: usar `JG` / `JL` (con signo) cuando el enunciado trabaja con datos `unsigned`, o usar `JA` / `JB` (sin signo) cuando los datos son `short` /con signo. El compilador a mano DEBE elegir el salto según el tipo de dato del enunciado, no “el que suena parecido”.

4.4 PILA Y CONTROL DE FLUJO (LAS INSTRUCCIONES DEL EJERCICIO ESTRELLA)

Instrucción	Efecto exacto	Flags
<code>PUSH src</code>	$SP \leftarrow SP - 2$; luego $[SS:SP] \leftarrow src$	ninguno
<code>POP dest</code>	$dest \leftarrow [SS:SP]$; luego $SP \leftarrow SP + 2$	ninguno
<code>CALL etiqueta (near)</code>	PUSH IP (dirección de retorno, 2 bytes); IP $\leftarrow$ dirección de la rutina	ninguno
<code>RET</code>	POP IP (retoma la dirección de retorno)	ninguno
<code>RET n</code>	Igual que RET, y además $SP \leftarrow SP + n$ (limpia n bytes de parámetros de la pila)	ninguno
<code>INT n</code>	PUSH FLAGS; PUSH CS; PUSH IP; salta a la rutina de interrupción n	ninguno (aunque la rutina invocada puede modificarlos)
<code>IRET</code>	POP IP; POP CS; POP FLAGS (deshace exactamente lo que hizo INT)	restaura FLAGS completo

Nota sobre `CALL far` (entre segmentos distintos, como el Ejercicio 4 del Práctico 8): además de IP, también hace `PUSH CS` antes, por lo que la dirección de retorno ocupa **4 bytes** (CS+IP) en vez de 2. Esto cambia la cuenta de stack. En el ejercicio recursivo de la sección 6 usamos `CALL near` (la función se llama a sí misma dentro del mismo segmento de código), que es el caso típico y el que hay que dominar primero.

Truco para no perderte con PUSH/POP: la pila SIEMPRE crece hacia direcciones **bajas**. Cada PUSH resta 2 a SP (para una palabra); cada POP suma 2. Si al final de una rutina el SP no volvió a su valor de entrada, hay un PUSH sin su POP (o viceversa) — literalmente el bug más común de este ejercicio.

5. Directivas de ensamblador

Directiva	Uso	Ejemplo
<code>EQU</code>	Define una constante simbólica (no reserva memoria, es un alias en tiempo de ensamblado)	<code>CANT_MATRICES EQU 10</code>
<code>DB</code>	Define Byte(s): reserva 1 byte por valor	<code>letras DB 'A', 'B', 'C'</code>
<code>DW</code>	Define Word(s): reserva 2 bytes por valor	<code>VECTOR DW 1024 DUP(0) (1024 palabras)</code>
<code>DD</code>	Define Doubleword: reserva 4 bytes por valor (p. ej. un puntero <i>far</i> : offset+segmento)	<code>ptr DD ?</code>
<code>byte ptr</code>	Fuerza a que el ensamblador interprete el tamaño del operando de memoria como 1 byte, cuando es ambiguo	<code>MOV BYTE PTR [DI], 0</code>
<code>word ptr</code>	Igual que arriba pero para 2 bytes	<code>MOV WORD PTR [BX], 0</code>
<code>PROC</code> / <code>ENDP</code>	Delimitan el cuerpo de un procedimiento/función	<code>factorial PROC ... factorial ENDP</code>

Ojo con “DDW”: esa directiva no existe en los ensambladores tipo MASM/TASM que usa este curso. Las tres directivas de definición de datos son `DB` (byte, 1B), `DW` (word, 2B) y `DD` (doubleword, 4B). Si ves “DDW” en un apunte o lo escribís en el parcial, es un error de tipeo por “DD” — corregilo así en tu cartilla.

¿Por qué hace falta `byte ptr` / `word ptr`? Una instrucción como `MOV [DI], 0` es ambigua: el ensamblador no sabe si tenés que escribir un byte (0x00) o una palabra (0x0000) en memoria, porque el inmediato “0” no tiene tamaño intrínseco y `[DI]` no lo indica por sí solo (a diferencia de `MOV AL, [DI]`, donde el tamaño de AL ya lo fija). Hay que decírselo explícitamente con `BYTE PTR` o `WORD PTR`. Vas a necesitar esto cada vez que el destino en memoria no tenga otro operando (registro) que fije el tamaño — literalmente pasa en el caso base de `ConstruirCadena` del Práctico 8: `mov byte ptr di, 0` (en notación estricta de MASM se escribe con corchetes: `MOV BYTE PTR [DI], 0`; algunos apuntes del curso omiten los corchetes de forma abreviada, pero conceptualmente es indispensable aclarar el tamaño).

6. EL EJERCICIO MÁS IMPORTANTE DEL CURSO: compilar a mano una función recursiva

6.1 EL PROBLEMA EN ALTO NIVEL

Vamos a compilar `factorial`, la función recursiva más simple posible, pero resolviéndola con el protocolo de pasaje de parámetros **por pila** (el caso que más rinde en examen, porque obliga a armar el frame con BP — a diferencia de pasar todo por registros, que evita ese paso):

```
short factorial(short n) {
    if (n <= 1)
        return 1;
    else
        return n * factorial(n - 1);
}
```

Protocolo elegido (típico de este curso, ver Práctico 7 Ejercicio 10 y Práctico 8 Ejercicio 4):

- El parámetro `n` (16 bits) se pasa en la pila: el llamador hace `PUSH n` antes del `CALL`.
- El resultado se devuelve en `AX`.
- La función preserva todos los demás registros que usa internamente (en este caso, `BX`).
- El llamador es responsable de retirar el parámetro de la pila después del `RET` (`ADD SP, 2`).

6.2 CÓDIGO ENSAMBLADOR COMPLETO, LÍNEA POR LÍNEA

```
; short factorial(short n)
; Protocolo: el llamador hace PUSH n ; CALL factorial ; y tras el RET hace ADD SP,2
; para retirar el parametro. Resultado en AX. BX se preserva.

factorial PROC
    PUSH BP                ; (1) salvamos el BP del llamador
    MOV BP, SP             ; (2) armamos el frame: [BP+0]=BP viejo, [BP+2]=dir. retorno, [BP+4]=n
    PUSH BX                ; (3) salvamos BX porque lo vamos a usar como variable de trabajo

    MOV BX, [BP+4]         ; (4) BX = n (parametro)
    CMP BX, 1
    JG _recursivo          ; (5) si n > 1, caso recursivo

_base:
    MOV AX, 1              ; (6) caso base: factorial(0) = factorial(1) = 1
    JMP _fin

_recursivo:
    DEC BX                 ; (7) BX = n - 1
    PUSH BX                ; (8) apilamos el parametro para la llamada recursiva
    CALL factorial          ; (9) factorial(n-1); al volver, resultado en AX
    ADD SP, 2              ; (10) retiramos el parametro (n-1) que pusimos en (8)

    MOV BX, [BP+4]         ; (11) recuperamos el n ORIGINAL de este frame (BX se pisó en la llamada)
    IMUL BX                ; (12) DX:AX = AX * BX -> con n chico, el resultado entra en AX

_fin:
    POP BX                 ; (13) restauramos BX (simetrico del push en (3))
    POP BP                 ; (14) restauramos BP del llamador (simetrico del push en (1))
    RET                    ; (15) retorna; el llamador debe hacer ADD SP,2 para retirar n
factorial ENDP
```

Revisión línea por línea (checklist de tolerancia cero):

- `PUSH BP` (1) ↔ `POP BP` (14): correcto, y en el orden inverso (LIFO).
- `PUSH BX` (3) ↔ `POP BX` (13): correcto, también en orden inverso, y `POP BX` ocurre **antes** que `POP BP` porque `BX` se apiló **después** de `BP` (última en entrar, primera en salir).
- El `PUSH BX` del paso (8) es el **parámetro** de la llamada recursiva, no un registro a preservar; por eso no tiene un `POP BX` emparejado — se retira con `ADD SP,2` en (10), tal como indica el protocolo (el llamador limpia lo que empujó).
- `[BP+4]` apunta al parámetro porque, al entrar a la función, la pila (de arriba hacia abajo) es: `BP` viejo (en `[BP+0]`), dirección de retorno de 2 bytes de un `CALL near` (en `[BP+2]`), y `n` (en `[BP+4]`).
- En (11) releemos `[BP+4]` en vez de confiar en `BX`, porque `BX` fue destruido por la llamada recursiva de (9) — es exactamente la razón por la que la solución oficial de `fibonacci` en el Práctico 8 hace `PUSH AX` antes de cada `CALL` recursivo: los registros no sobreviven una llamada recursiva a menos que vos te encargues de salvarlos.

6.3 CÁLCULO DEL CONSUMO MÁXIMO DE STACK (N=5)

Fórmula clave:

Consumo de pila = (bytes de dirección de retorno + bytes de registros salvados + bytes de parámetros) × profundidad máxima de recursión

Para esta función, por cada activación (cada vez que se entra a `factorial`, sea la llamada inicial o una recursiva):

Componente	Bytes
Parámetro <code>n</code> apilado por el llamador antes del CALL	2
Dirección de retorno que apila el <code>CALL</code> (near, dentro del mismo segmento)	2
<code>PUSH BP</code> (frame del llamado)	2
<code>PUSH BX</code> (registro de trabajo salvado)	2
Total por activación	8 bytes

Con `n=5`, la cadena de llamadas es `factorial(5) → factorial(4) → factorial(3) → factorial(2) → factorial(1)`, es decir, **5 activaciones simultáneas** en el peor momento (justo cuando se está ejecutando el caso base `factorial(1)`, antes de que empiece a retornar).

Consumo máximo = 8 bytes/activación × 5 activaciones = 40 bytes

Verificación por simulación explícita de la pila (SP relativo, arrancando en 0 antes de que el programa principal invoque a `factorial`):

Paso	Acción	SP acumulado
1	PUSH 5 (parámetro, llamada externa)	2
2	CALL factorial(5) → push dir. retorno	4
3	[dentro] PUSH BP	6
4	[dentro] PUSH BX (BX=5)	8
5	PUSH 4, CALL factorial(4)	12
6	PUSH BP; PUSH BX (BX=4)	16
7	PUSH 3, CALL factorial(3)	20
8	PUSH BP; PUSH BX (BX=3)	24
9	PUSH 2, CALL factorial(2)	28
10	PUSH BP; PUSH BX (BX=2)	32
11	PUSH 1, CALL factorial(1)	36
12	PUSH BP; PUSH BX (BX=1) — caso base, no apila más	40

Máximo SP = 40 bytes. Coincide exactamente con $8 \times 5 = 40$. Verificado también con `bash: (2+4+2)*5 = 40`. ✓

Fórmula general para N: consumo máximo = $8 \times N$ bytes (para esta función en particular, donde cada activación cuesta 8 bytes hijos y la profundidad de recursión es igual a N, ya que `factorial` decrementa de a uno hasta el caso base).

Error frecuente (el que más resta puntos): confundir “cantidad de llamadas recursivas” con “profundidad máxima simultánea”. No son lo mismo en todas las funciones: en `factorial` o `fibonacci` con recursión simple (una sola rama activa a la vez) coinciden, pero en funciones con dos llamadas recursivas seguidas (como `ConstruirCadena` no, pero sí `fibonacci`) propiamente dicho, que llama primero a la izquierda y LUEGO a la derecha) hay que analizar cuál de las dos ramas genera la pila más profunda, no sumar todas las llamadas totales. En `fibonacci(n)` la pila máxima se alcanza siguiendo la primera rama izquierda hasta el fondo (ver 6.4), no en el total de llamadas del árbol de recursión completo.

6.4 SEGUNDO EJEMPLO RESUELTO Y VERIFICADO: `FIBONACCI` CON PARÁMETROS POR REGISTRO (PROTOCOLO ALTERNATIVO)

Este es el ejercicio 2 del Práctico 8, resuelto oficialmente por la cátedra, y sirve para contrastar un protocolo **sin** pila de parámetros (todo por registros, sin frame de BP):

```
short fibo(short n) {
    if (n < 2) return n;
    else return fibo(n-1) + fibo(n-2);
}
```

Protocolo: `n` se recibe en `AX`, el resultado se devuelve en `BX`.

```
fibonacci PROC
_if1:
    CMP AX, 2
    JGE _else
    MOV BX, AX          ; caso base: fibo(n) = n para n<2
    JMP _return
_else:
    PUSH AX             ; salvamos n, porque se pierde en la llamada recursiva
    DEC AX
    CALL fibonacci      ; BX <- fibo(n-1)
    POP AX              ; restauramos n (AX se destruyó dentro de la recursión)
    PUSH BX             ; salvamos fibo(n-1)
    SUB AX, 2
    CALL fibonacci      ; BX <- fibo(n-2)
    POP AX              ; AX <- fibo(n-1) (lo que habíamos guardado)
    ADD BX, AX          ; BX <- fibo(n-2) + fibo(n-1)
_return:
    RET
fibonacci ENDP
```

Cálculo de stack para n=5 (verificado por la cátedra):

- Invocación inicial (sin `PUSH AX` porque la llama el programa principal, no la propia función): solo la dirección de retorno → **2 bytes**.
- Cada llamada recursiva “hacia la izquierda” (`fibo(n-1)`) hace `PUSH AX` + `CALL` → **4 bytes**.
- La rama que llega más profundo antes de un caso base es siempre la rama izquierda (`n-1, n-1, n-1, ...`), y para n=5 hacen falta **4 llamadas recursivas** para llegar a `fibo(1)` (caso base): `fibo(5)→fibo(4)→fibo(3)→fibo(2)→fibo(1)`.

Consumo máximo = invocación (2) + 4 llamadas recursivas × 4 bytes = 2 + 16 = 18 bytes

Verificado independientemente: $2 + 4 \times 4 = 18$. Coincide con el resultado oficial de la cátedra (“el máximo tamaño que toma la pila es de 9 palabras, o sea, 18 bytes”). ✓

Compará este resultado (18 bytes) con el de nuestro `factorial` por pila (40 bytes) para el mismo n=5: pasar parámetros por registro en vez de por pila reduce drásticamente el consumo de stack, porque te ahorrás 2 bytes de parámetro y 4 bytes de frame (BP+BX) por cada activación. Esta comparación —“¿qué protocolo consume menos pila?”— es un tipo de pregunta de parcial en sí misma.

7. Por qué `[BX+SI]` (direccionamiento indexado) es clave para recorrer arreglos y arrays de structs

7.1 LA IDEA

Cuando recorres un arreglo, necesitas una dirección que combine un **puntero fijo al inicio de la estructura** (que no cambia mientras recorres) con un **desplazamiento variable que avanza** con cada elemento. El modo `[BX+SI]` (o `[BX+DI]`, `[BP+SI]`, `[BP+DI]`) te da exactamente eso con una sola instrucción de memoria, en vez de tener que sumar y volver a cargar un puntero completo en cada iteración.

- `BX` (o `BP`) hace de **puntero base**: la dirección donde arranca el arreglo/estructura.
- `SI` (o `DI`) hace de **índice**: cuánto avanzaste, típicamente `índice × tamaño_del_elemento`.

7.2 EJEMPLO: ARRAY DE STRUCTS (PRÁCTICO 7, EJERCICIO 7 / PRÁCTICO 8, `ARBOL`)

```
struct mi_tipo {
    char letras[4];      // 4 bytes
    short enteros[2];    // 4 bytes
    char letras_MAYUS[2]; // 2 bytes
};                      // total: 10 bytes por elemento
mi_tipo mi_var[3];      // 30 bytes en total
```

Si `BX` apunta al comienzo de `mi_var` (offset dentro de DS) y quieres acceder al struct `i`, la dirección del elemento `i` es `BX + i*10`. Si tienes el índice `i` en `SI` **ya multiplicado por 10** (el tamaño del struct), entonces `[BX+SI]` te da directamente el comienzo del struct `i` sin recalcular nada:

```
; SI = i * 10 (previamente calculado)
MOV AX, [BX+SI]      ; letras[0..1] del elemento i
MOV DX, [BX+SI+4]    ; enteros[0] del elemento i (offset +4 dentro del struct)
```

Esto es exactamente el patrón que usa la solución oficial de `busquedaBin` (Práctico 8, ejercicio 3), donde `arbol` es un arreglo de structs `Nodo` de 4 bytes cada uno, y el índice recibido se convierte a dirección de memoria con `SHL BX, 2` (equivalente a `× 4`, el tamaño del struct) antes de indexar con `ES:[BX]`.

Truco general: si el tamaño del struct/elemento es potencia de 2 (2, 4, 8, 16...), multiplicar el índice por el tamaño se hace con `SHL` (más rápido que `MUL`). Si no es potencia de 2 (por ejemplo 10, como en `mi_tipo`), hay que usar sumas/`MUL` explícito, como en el ejemplo de `ConstruirCadena`: `mov ax, bx; add ax, ax; add bx, ax` (esto calcula `bx = bx*3` combinando duplicar y sumar, porque cada 'item' de esa lista ocupa 3 bytes).

7.3 EJEMPLO: MATRIZ DE MATRICES (PRÁCTICO 7, EJERCICIO 9)

Para recorrer un arreglo de matrices 4×4 de enteros (cada matriz ocupa 4×4×2 = 32 bytes), el mismo patrón aplica en dos niveles: un índice para "qué matriz" (multiplicado por 32) y, dentro de esa matriz, otro índice para "qué elemento" (multiplicado por 2, el tamaño de un `short`). Direccionamiento `[BX+SI]` con `BX` apuntando a la matriz actual y `SI` recorriendo sus 16 elementos consecutivos es la forma estándar de resolverlo sin recalcular la dirección base en cada paso del bucle interno.

Error frecuente: olvidar multiplicar el índice lógico por el tamaño del elemento antes de sumarlo como offset. `[BX+SI]` con `SI = i` (índice lógico sin escalar) en vez de `SI = i×tamaño` da una dirección completamente incorrecta si el elemento ocupa más de 1 byte.

Ejercicios de repaso rápido (con respuesta)

Pregunta	Respuesta
Dirección física de <code>0x1000:0x0010</code>	<code>10010h</code> ($1000h \times 16 + 10h$)
¿Qué segmento se usa por defecto con <code>[BP+4]</code> sin prefijo?	SS (la pila)
¿Qué flags afecta <code>NOT AX</code> ?	Ninguno
¿Qué salto usás para “mayor que” con datos <code>unsigned char</code> ?	<code>JA</code> (o <code>JNBE</code>)
¿Qué salto usás para “mayor o igual” con datos <code>short</code> (con signo)?	<code>JGE</code> (o <code>JNL</code>)
<code>MUL BX</code> con AX y BX de 16 bits, ¿dónde queda el resultado?	En <code>DX:AX</code> (DX = mitad alta, AX = mitad baja)
¿Qué instrucción arma el frame de una función junto con <code>MOV BP,SP</code> ?	<code>PUSH BP</code> (antes), y se deshace con <code>POP BP</code> (al final)
Si una función <code>naar</code> recibe un parámetro de 16 bits por pila y arma frame con BP, ¿en qué offset respecto a BP está el parámetro?	<code>[BP+4]</code> (BP viejo en +0, dirección de retorno <i>near</i> de 2 bytes en +2, parámetro en +4)
Para <code>factorial(n)</code> con parámetro por pila + frame BP + 1 registro salvado, ¿cuántos bytes consume cada activación?	8 bytes (2 parámetro + 2 retorno + 2 BP + 2 registro)
Consumo máximo de pila de <code>factorial</code> (protocolo de la sección 6) para n=5	40 bytes
Consumo máximo de pila de <code>fibonacci</code> por registros (sección 6.4) para n=5	18 bytes
¿ <code>DDW</code> es una directiva válida de MASM/TASM?	No — es <code>DD</code> (doubleword, 4 bytes)
¿Qué instrucción hay que usar para desambiguar el tamaño en <code>MOV [DI], 0</code> ?	<code>BYTE PTR</code> o <code>WORD PTR</code> antes del operando de memoria
¿Qué banderas hay que consultar para detectar overflow con signo en una suma?	OF (Overflow Flag)
¿Qué banderas hay que consultar para detectar overflow sin signo (acarreo) en una suma?	CF (Carry Flag)

Resumen ultra-rápido (2 minutos)

- **Dirección física = SEGMENTO×16 + DESPLAZAMIENTO.** Agregale un `0` hex al segmento antes de sumar. Distintos pares `SEG:OFF` pueden dar la misma dirección física (segmentos solapados).
- **Registros:** AX/BX/CX/DX (+ mitades H/L), segmento CS/DS/SS/ES, SP y BP (pila, viven en SS por defecto), SI y DI (índices, viven en DS/ES por defecto), IP (no tocable directo), FLAGS (ZF, SF, CF, OF son las que importan).
- **Modos de direccionamiento:** registro, inmediato, directo, `[reg]`, `[BX+SI]`, `[BP+DI+desp]`. Regla del segmento por defecto: si interviene BP → SS; si no → DS (salvo prefijo explícito).
- **Instrucciones clave y sus flags:** aritméticas (ADD/SUB) tocan ZF/SF/CF/OF; lógicas (AND/OR/XOR) tocan ZF/SF y limpian CF/OF; NOT no toca flags; saltos con signo (JG/JL/JGE/JLE) miran SF y OF; saltos sin signo (JA/JB/JAE/JBE) miran CF y ZF.
- **PUSH resta 2 a SP y escribe; POP lee y suma 2 a SP.** CALL *near* apila IP (2 bytes) y RET lo recupera. Cada PUSH necesita su POP simétrico (orden LIFO).
- **Directivas:** EQU (constante), DB/DW/DD (1/2/4 bytes — “DDW” no existe), BYTE PTR/WORD PTR (desambiguar tamaño), PROC/ENDP (delimitar función).
- **El ejercicio estrella:** armar el frame con `PUSH BP` + `MOV BP,SP`, acceder a parámetros en `[BP+4]`, `[BP+6]` ... (según cuántos bytes se apilaron antes), salvar registros de trabajo con PUSH/POP simétricos, y calcular el consumo de pila como **(dirección de retorno + registros salvados + parámetros) × profundidad máxima de recursión**. Para funciones con dos llamadas recursivas (como fibonacci), la profundidad máxima es la de la rama que llega primero a un caso base (normalmente la izquierda), no la suma de todas las llamadas del árbol.